# 课 程 内 容

第 1 章 编 译 引 论

第 2 章 形式语言与自动机基础

第 3 章 词法分析 (Lexical Analysis)

第 4 章 语法分析(Syntax Analysis)
—— 自上而下分析法

第 5 章 语法分析(Syntax Analysis)
—— 自下而上分析法

第 6 章 语义分析与中间代码生成

第 7 章 运 行 环 境

第 8 章 代码优化（optimization）

# 第 4 章　语法分析(Syntax Analysis)

## —— 自上而下分析法

4.1　语法分析综述

4.2　不确定的自上而下分析法

4.3　递归下降分析法与递归下降分析器

4.4　LL(1)分析法与LL(1)分析器

# 4.1 语法分析综述

4.1.1 语法分析程序的功能

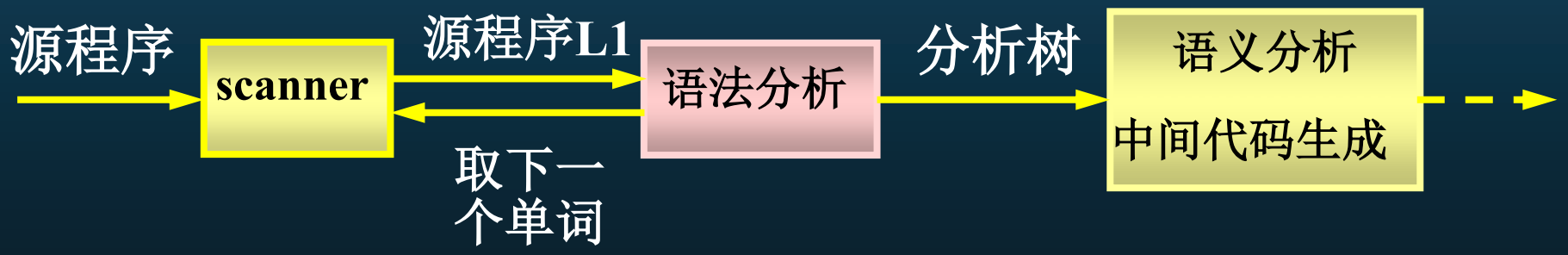
4.1.2 语法分析方法

一. 自上而下分析

二. 自下而上分析

## 语法分析程序的功能

完成语法分析任务的程序称为语法分析器，或语法分析程序。



## 语法分析程序的功能

按照源语言的语法规则，从词法分析的结果中识别出相应的语法范畴，同时进行语法检查。

给定文法$G$和字符串$\alpha$( $\alpha \in V_T^*$)，检查、判定 $\alpha \in L(G)$？即检查、判定$\alpha$是否是文法$G$所能产生的合法的句子，同时报告和处理语法错误。

- 语法分析程序的构造要素

源程序串（L1形式） ⟶ 处理对象

源语言的文法*G* ⟶ 分析依据

识别出的语法范畴的表示 ⟶ 分析结果

# 4.1 语法分析综述

## 4.1.1 语法分析程序的功能

## 4.1.2 语法分析方法

一. 自上而下分析

二. 自下而上分析

# 一．自上而下语法分析方法

给定文法*G*和源程序串*r*。从*G*的开始符号*S*出发，通过反复使用产生式对句型中的非终结符进行替换(推导)，逐步推导出*r* 。

* 是一种产生的方法，面向目标的方法。
* 分析的主旨是选择产生式的合适的侯选式进行推导，逐步使推导结果与*r*匹配。

例4.1 设有文法$G$和输入串 $r$

$G$: $S \rightarrow aA$

$A \rightarrow BaA \mid \varepsilon$

$B \rightarrow + \mid - \mid * \mid ,$

$r$: $a*a+a$

分析方式1(推导):

$S \Rightarrow \underline{aA} \Rightarrow a\underline{BaA} \Rightarrow a\underline{*}aA \Rightarrow a*a\underline{BaA}$

$\Rightarrow a*a\underline{+}aA \Rightarrow a*a+a = r \in L(G)$

# 分析树的从左到右叶结点=$r$，则 $r \in L(G)$。

$S \rightarrow aA$

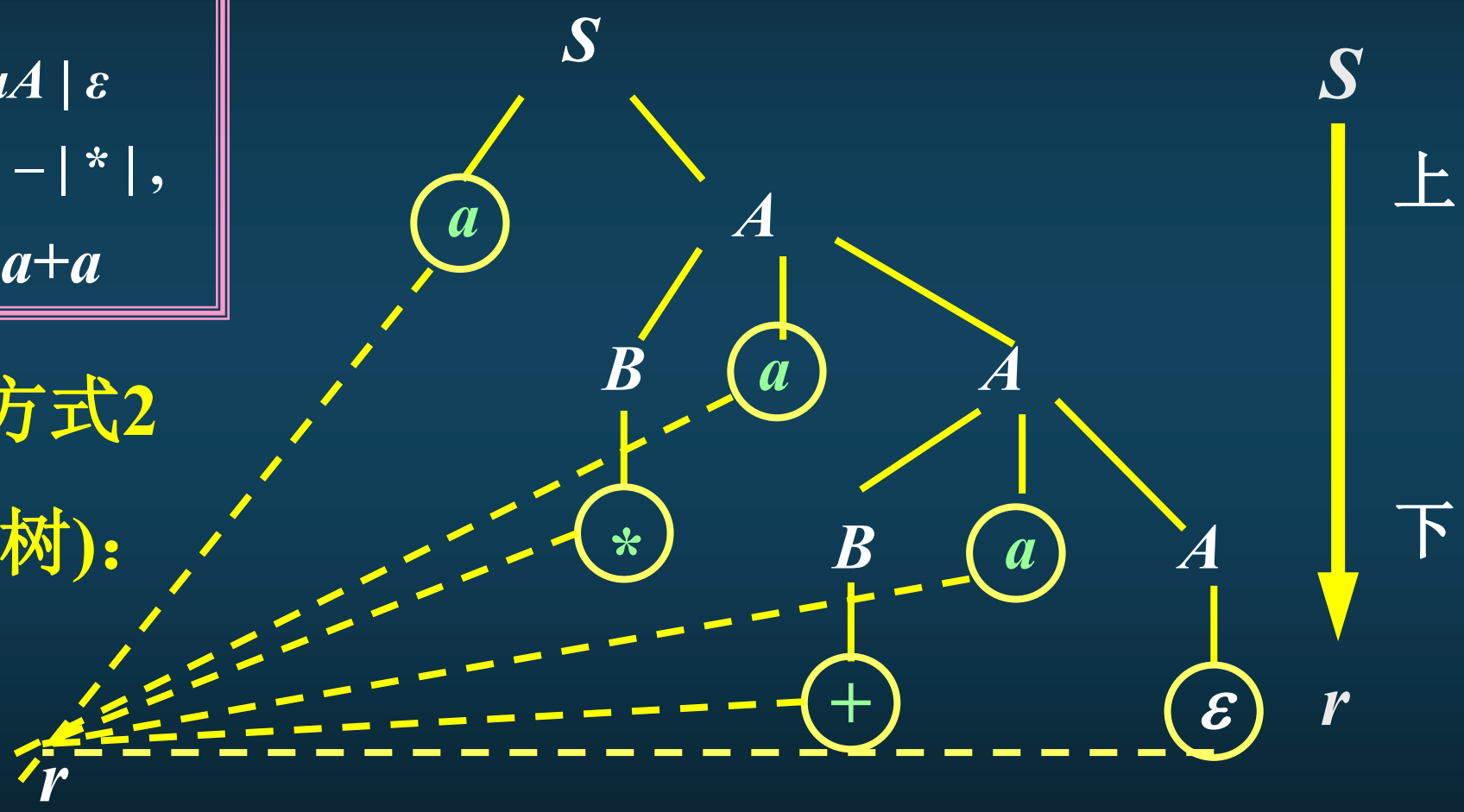
$A \rightarrow BaA \mid \varepsilon$

$B \rightarrow + \mid - \mid * \mid ,$

$r$: $a*a+a$

分析方式2

(分析树):



# 4.1 语法分析综述

## 4.1.1 语法分析程序的功能

## 4.1.2 语法分析方法

一. 自上而下分析

二. 自下而上分析

## 二．自下而上语法分析方法

从给定的输入串*r*开始，不断寻找子串与文法*G*中某个产生式*P*的候选式进行匹配，并用*P*的左部代替(归约)之，逐步归约到开始符号*S*。

* 是一种辨认的方法，基于目标的方法。
* 分析的主旨是寻找合适的子串与*P*的侯选式进行匹配，直到归约到*G*的*S*为止 。

# 一．自上而下语法分析方法

给定文法*G*和源程序串*r*。从*G*的开始符号*S*出发，通过反复使用产生式对句型中的非终结符进行替换(推导)，逐步推导出*r* 。

* 是一种产生的方法，面向目标的方法。
* 分析的主旨是选择产生式的合适的侯选式进行推导，逐步使推导结果与r匹配。

## 二．自下而上语法分析方法

从给定的输入串*r*开始，不断寻找子串与文法*G*中某个产生式*P*的候选式进行匹配，并用*P*的左部代替(归约)之，逐步归约到*S*。

* 是一种辨认的方法，基于目标的方法。
* 分析的主旨是寻找合适的子串与*P*的侯选式进行匹配，直到归约到*G*的*S*为止 。

例4.2 设有文法$G$和输入串 $r$

$G$: $S \rightarrow aA$

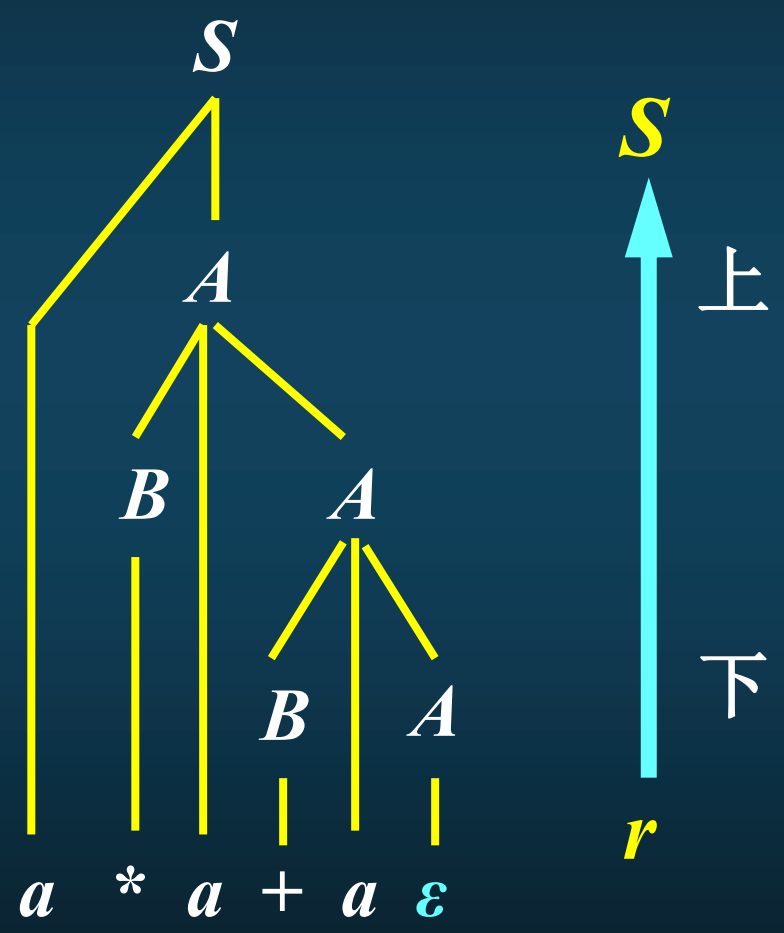
$A \rightarrow BaA \mid \varepsilon$

$B \rightarrow + \mid - \mid * \mid ,$

$r$: $a*a+a$

分析方式1

(分析树):



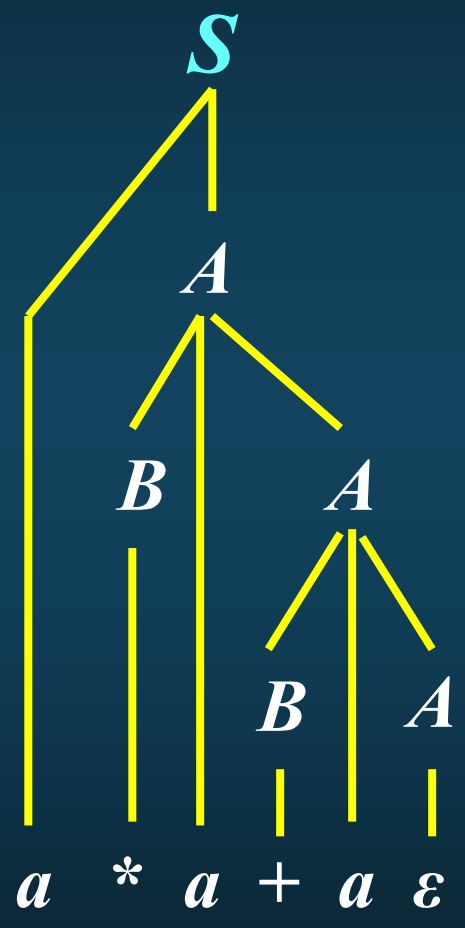
## 例4.2 设有文法G和输入串 r

G: $S \rightarrow aA$

$A \rightarrow BaA \mid \varepsilon$

$B \rightarrow + \mid - \mid * \mid ,$

r: $a*a+a$




## 分析方式2(归约):

$r = a*a+a \Leftarrow a\underline{B}a+a$

$\Leftarrow aBa\underline{B}a\varepsilon \Leftarrow aBaBa\underline{A}$

$\Leftarrow aBa\underline{A} \Leftarrow a\underline{A} \Leftarrow S$

## 说明：

1. 语法分析两大类方法：

自上而下分析法；$S \longrightarrow r$ (一般、递归下降、LL(1))

自下而上分析法；$r \xrightarrow{\text{Reduce}} S$ (一般$S$-$R$、OPG、LR)

2. 自上而下分析法的核心是不断寻找合适候选式对句型中最左的非终结符进行替换的过程；

3. 自下而上分析法的核心是不断寻找可归约串与$P$的侯选式匹配，并用$P$的左部的非终结符代替之。

# 第 4 章　语法分析(Syntax Analysis)

## —— 自上而下分析法

4.1　语法分析综述

4.2　不确定的自上而下分析法

4.3　递归下降分析法与递归下降分析器

4.4　LL(1)分析法与LL(1)分析器

# 4.2 不确定的自上而下分析法

4.2.1 一般自上而下分析

4.2.2 不确定性的原因及解决方法

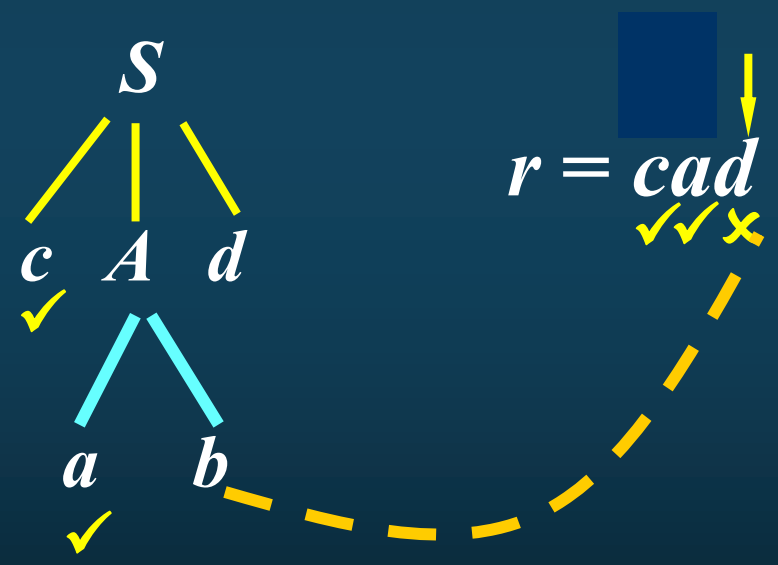
# 例4.4 设有如下文法 $G$和字符串 $r = cad$

(1) $S \rightarrow cAd$

(2) $A \rightarrow ab \mid a$

**Step1:**

**Step2:**

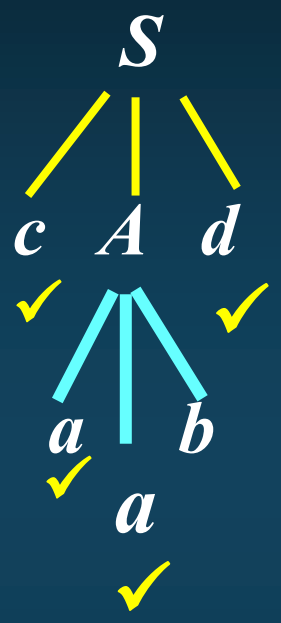


(1) $S \rightarrow cAd$

(2) $A \rightarrow ab \mid a$

**Step3:**

**Step4:**




$r = cad$

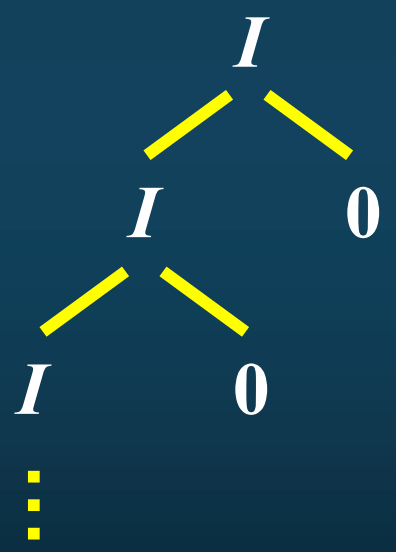
分析的本质是一种带回溯的自上而下分析,是一试探推导的过程，即反复使用不同的产生式去谋求匹配输入串的过程。算法效率低开销大。

# 例4.5 设有文法 $G$和输入字符串 $r$

$G$: $I \rightarrow I0 \mid Ia \mid a$

$r$ : $a00$

$L(G)=a(a|0)^*$



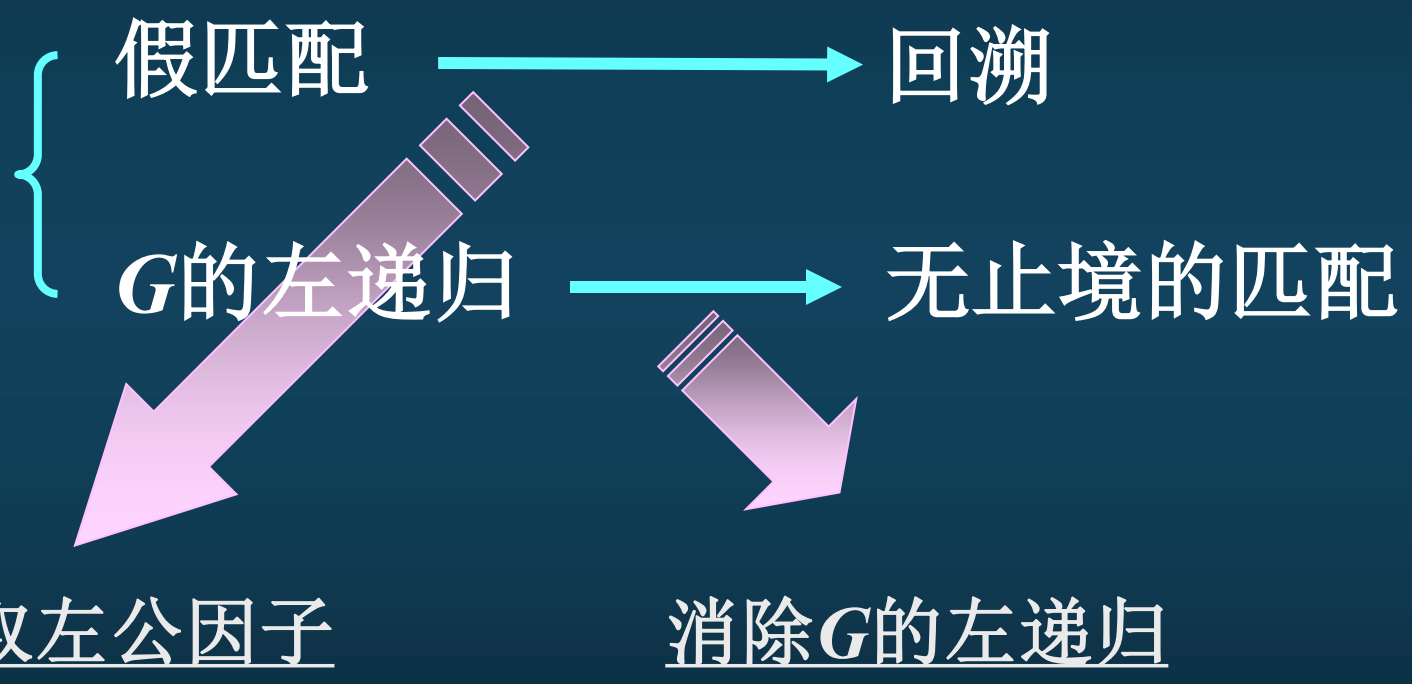

按照自上而下分析法对输入串$r$产生分析树，对非终结符的替换使分析树无休止的延伸，使自上而下分析陷入死循环。

# 4.2 不确定的自上而下分析法

4.2.1 一般自上而下分析

4.2.2 不确定性的原因及解决方法

# 不确定性的原因



# 一. 消除文法的左递归

文法的直接左递归表现在其含有$A \rightarrow A\alpha$（$\alpha \in (V_T \cup V_N)^*$）形式的产生式规则，则在语法分析的最左推导中会呈现$A \Rightarrow A$……的形式，间接左递归文法会呈现$A \overset{+}{\Rightarrow} A$…… 的形式。

# （1） 直接左递归的消除：

假定关于非终结符$P$的规则为

$$P \rightarrow P\alpha \mid \beta \quad \alpha、\beta \in (V_T \cup V_N)^*$$

其中，$\beta$不以$P$开头。

分析： $P \rightarrow P\alpha \mid \beta$ → $L(G(P))=\{\beta\alpha^*\}$

不变 → $\beta$

个数可以无穷多个，只能用 递归表示且不 能用左递归。 → $P'$

$$P \rightarrow \beta P'$$

$$P' \rightarrow \alpha P' \mid \varepsilon$$

## （1） 直接左递归的消除：

假定关于非终结符$P$的规则为

$$P \rightarrow P\alpha \mid \beta \quad \alpha、\beta \in (V_T \cup V_N)^*$$

其中，$\beta$不以$P$开头。

把$P$的规则改写成如下 等价的非直接左递归形式

$$P \rightarrow \beta P'$$

$$P' \rightarrow \alpha P' \mid \varepsilon$$

**例4.6** 设有简单表达式文法$G(E)$：

$$E \rightarrow E+E \mid E*E \mid (E) \mid i$$

对$G(E)$消除二义性后，得到文法$G'(E)$：

$\alpha$

$$E \rightarrow E+T \mid T$$
$$T \rightarrow T*F \mid F$$
$$F \rightarrow (E) \mid i$$

$\beta$

继续消除文法$G'(E)$的左递归，得到文法$G''(E)$

$$E \rightarrow TE'$$
$$E' \rightarrow +TE' \mid \varepsilon$$
$$T \rightarrow FT'$$
$$T' \rightarrow *FT' \mid \varepsilon$$
$$F \rightarrow (E) \mid i$$

假定关于非终结符$P$的规则为

$$P \rightarrow P\alpha_1 | P\alpha_2 | \ldots | P\alpha_n | \beta_1 | \beta_2 | \ldots | \beta_n$$

其中，每个$\alpha_i$不等于$\varepsilon$，$\beta_1$ ,$\beta_2$ ,…,$\beta_n$ 不以$P$开头。则可以把$P$的规则改写成如下等价的非直接左递归形式

$$P \rightarrow \beta_1 P' | \beta_2 P' | \ldots | \beta_n P'$$

$$P' \rightarrow \alpha_1 P' | \alpha_2 P' | \ldots | \alpha_n P' | \varepsilon$$

**例4.7** 设有文法$G$:

$$I \rightarrow I0 \mid Ia \mid Ib \mid a \mid b$$

对左递归文法$G$改写后的文法$G'$为

$$I \rightarrow aI' \mid bI'$$

$$I' \rightarrow 0I' \mid aI' \mid bI' \mid \varepsilon$$

## （2） 间接左递归的消除：

有些文法表面上不具有左递归性，却隐含着左递归。例如设有文法$G(A)$：

$$A \rightarrow Ba \mid a$$

$$B \rightarrow Cb \mid b$$

$$C \rightarrow Ac \mid c$$

经若干步推导替换，有：

$$A \Rightarrow Ba \Rightarrow Cba \Rightarrow Acba$$

$$B \Rightarrow Cb \Rightarrow Acb \Rightarrow Bacb$$

$$C \Rightarrow Ac \Rightarrow Bac \Rightarrow Cbac$$

- 消除间接左递归的方法：

  (1) 把间接左递归文法改写为直接左递归文法；

  (2) 用消除直接左递归的方法改写文法。

后面给出一个消除文法所有左递归性的算法，该算法对文法的要求是，文法不含回路（形如 $P \overset{+}{\Rightarrow} P$的推导）且不含以$\varepsilon$为右部的产生式。

**思考题：**

任何一文法都等价于一个不含回路且不含以$\varepsilon$为右部的产生式的文法。

- **算法4.1（消除文法左递归）**

给定文法$G$

① 对文法$G$的所有非终结符按任一种顺序排列，例如$A_1$，$A_2$，…，$A_n$。消除$A_1$中的直接左递归。

② for ( i=1;i=n;i++ )

{for ( j=1; j = i-1;j++ )

{

把形如$A_i \rightarrow A_j\gamma$ 的产生式改写成

$A_i \rightarrow \delta_1\gamma \mid \delta_2\gamma \mid \ldots\ldots \mid \delta_k\gamma$

其中$A_j \rightarrow \delta_1 \mid \delta_2 \mid \ldots\ldots \mid \delta_k$ 是关于$A_j$的全部规则；

}

消除$A_i$规则中的直接左递归；}

③ 简化由②所得的文法，即去掉多余的规则。

令文法$G(A)$的非终结符排序为$C$，$B$，$A$。

对于$C$，不存在直接左递归，把$C$代入$B$，$B$的规则变为

$$B \rightarrow Acb \mid dAb \mid c$$

$$G(A): A \rightarrow Ba \mid bB$$
$$B \rightarrow Cb \mid c$$
$$C \rightarrow Ac \mid dA$$

代换后的$B$不含直接左递归，将其代入$A$，$A$的规则变为：

$$A \rightarrow Acba \mid dAba \mid ca \mid bB$$

$A$存在直接左递归，消除$A$的直接左递归有

$$A \rightarrow dAbaA' \mid caA' \mid bBA'$$

$$A' \rightarrow cbaA' \mid \varepsilon$$

文法$G(A)$改写为

$$A \rightarrow dAbaA' \mid caA' \mid bBA'$$

$$A' \rightarrow cbaA' \mid \varepsilon$$

$$B \rightarrow Acb \mid dAb \mid c$$

$$\boxed{C \rightarrow Ac \mid c}$$

删除多余的规则，最后得到文法$G(A)$的无左递归的等价文法$G'(A)$ 为

$$A \rightarrow dAbaA' \mid caA' \mid bBA'$$

$$A' \rightarrow cbaA' \mid \varepsilon$$

$$B \rightarrow Acb \mid dAb \mid c$$

令文法$G(A)$的非终结符排序为$A$，$B$，$C$。

对于$A$，不存在直接左递归，

对于$B$，候选式中不包含$A$，且不存在直接左递归。

$$G(A): A \rightarrow Ba \mid bB$$
$$B \rightarrow Cb \mid c$$
$$C \rightarrow Ac \mid dA$$

将$A$，$B$带入$C$，代换后的$C$的规则变为：

$$C \rightarrow Bac \mid bBc \mid dA$$
$$C \rightarrow Cbac \mid cac \mid bBc \mid dA$$

$C$存在直接左递归，消除$C$的直接左递归有

$$C \rightarrow cacC' \mid bBcC' \mid dAC'$$
$$C' \rightarrow bacC' \mid \varepsilon$$

文法$G(A)$改写为

$$A \rightarrow Ba \mid bB$$

$$B \rightarrow Cb \mid c$$

$$C \rightarrow cacC' \mid bBcC' \mid dAC'$$

$$C' \rightarrow bacC' \mid \varepsilon$$

## 二. 消除回溯

$P \rightarrow \alpha_1 \mid \alpha_2 \mid \ldots \mid \alpha_n$ 当前$r$: $\ldots a_i \ldots$

在一般自上而下分析中，对于一个$V_N$进行推导并试图去匹配句子剩余符号时，若$V_N$含有两个或两个以上的候选式，是依次一个一个地去试探，试图找出一个合乎要求的$\alpha_i$。先选$\alpha_1$，与当前输入$a_i$匹配成功则替换，否则选$\alpha_2$，依此类推。

## 定义4.4

设$G$是二型文法，则$G$中的任意$\alpha \in V^*$的终结首符集FIRST($\alpha$)为

$$\text{FIRST}(\alpha) = \{ a \mid \alpha \overset{*}{\Rightarrow} a \ldots\ldots,\ a \in V_T \}$$

若 $\alpha \overset{*}{\Rightarrow} \varepsilon$, 则$\varepsilon \in$ FIRST($\alpha$) 。

**不带回溯的条件：(充分非必要)**

如果对文法$G$的关于每一个非终结符$A$的产生式，设为：

$$A \rightarrow \alpha_1 \mid \alpha_2 \mid \ldots \mid \alpha_n$$

如果它的每个候选式$\alpha_i$均不存在$\alpha_i \overset{*}{\Rightarrow} \varepsilon$的情况，而且FIRST($\alpha_i$)两两彼此互不相交。

如果对文法$G$的关于每一个非终结符$A$的产生式，设为：

$$A \rightarrow \alpha_1 \mid \alpha_2 \mid \ldots \mid \alpha_n$$

满足不带回溯条件

则据当前扫描单词$a$，若$a \in \mathrm{FIRST}(\alpha_i)$，其中$\alpha_i$是$\alpha_1 \ldots \alpha_n$中之一，选取$A \rightarrow \alpha_i$进行推导。

# 计算FIRST($X$)($X\in V$)的算法描述：

为构造FIRST($X$)， 可反复应用如下规则：

1、若$X$是终结符,则FIRST($X$)＝{$X$};

2、若$X$是非终结符，$X$的FIRST为其所有候选式的FIRST集合的并集。

# 计算FIRST($\alpha$)($\alpha \in V^*$)的算法描述：

设$\alpha = X_1X_2\ldots X_k$ ($X_i \in V$)

为构造FIRST($\alpha$)， 可反复应用如下规则：

1、 FIRST($X_1$)中的所有终结符号加到FIRST($\alpha$)中；

2、若对于某个$i$，$a \in$ FIRST($X_i$)且$X_1X_2\ldots X_{i-1} \Rightarrow \varepsilon$，
则将$a$加到FIRST($\alpha$)中；

3、若对于所有的$j$＝1，2，…，$k$， $\varepsilon \in$ FIRST($X_j$)，
则将$\varepsilon$加到FIRST($\alpha$)中。

**例4.8** 设有文法$G$:

$$S \rightarrow Ap \mid Bq$$

$$A \rightarrow a \mid cA$$

$$B \rightarrow b \mid dB$$

对$S$: FIRST($Ap$)={$a$,$c$} FIRST($Bq$)={$b$,$d$}

FIRST($Ap$) ∩ FIRST($Bq$)=$\Phi$

对$A$: FIRST($a$)={$a$} FIRST($cA$)={$c$}

FIRST($a$) ∩ FIRST($cA$)=$\Phi$

对$B$: FIRST($b$)={$b$} FIRST($dB$)={$d$}

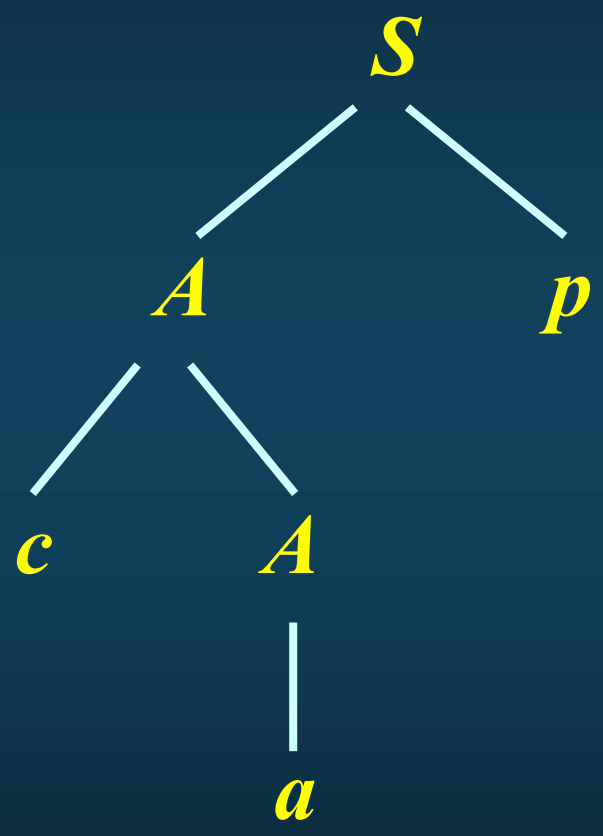
FIRST($b$) ∩ FIRST($dB$)=$\Phi$

$S \rightarrow Ap \mid Bq$

$A \rightarrow a \mid cA$

$B \rightarrow b \mid dB$

若给出$r=cap$ ,则有：




$r$: $cap$

$r$: $cap$

$r$: $cap$

FIRST($Ap$)={$a$,$c$}

FIRST($Bq$)={$b$,$d$}

FIRST($a$)={$a$}

FIRST($cA$)={$c$}

FIRST($b$)={$b$}

FIRST($dB$)={$d$}

对非终结符$A$的多个候选式的FIRST($\alpha_i$)的相互两个彼此交集$\neq\Phi$，一般是因为$\alpha_i$中有公共左因子，可以通过提取左公因子来改造文法。(由BNF范式改EBNF范式)

若有文法$G$:

$$A \rightarrow \delta\beta_1 \mid \delta\beta_2 \mid \ldots \mid \delta\beta_n$$

$$\delta(\beta_1 \mid \beta_2 \mid \ldots \mid \beta_n)$$

可以使用下面规则改写文法$G$为$G'$:

$$A \rightarrow \delta A'$$

$$A' \rightarrow \beta_1 \mid \beta_2 \mid \ldots \mid \beta_n$$

若文法$G$为：

$$A \rightarrow \underbrace{\delta_1\beta_1 \mid \delta_1\beta_2 \mid \ldots \mid \delta_1\beta_n}_{\delta_1(\underbrace{\beta_1 \mid \beta_2 \mid \ldots \mid \beta_n}_{A'})} \mid \underbrace{\delta_2\alpha_1 \mid \delta_2\alpha_2 \mid \ldots \mid \delta_2\alpha_m}_{\delta_2(\underbrace{\alpha_1 \mid \alpha_2 \mid \ldots \mid \alpha_m}_{A''})}$$

使用下面规则改写文法$G$为$G'$：

$$A \rightarrow \delta_1 A' \mid \delta_2 A''$$

$$A' \rightarrow \beta_1 \mid \beta_2 \mid \ldots \mid \beta_n$$

$$A'' \rightarrow \alpha_1 \mid \alpha_2 \mid \ldots \mid \alpha_m$$

# 第 4 章　语法分析(Syntax Analysis)

## —— 自上而下分析法

4.1　语法分析综述

4.2　不确定的自上而下分析法

4.3　递归下降分析法与递归下降分析器

4.4　LL(1)分析法与LL(1)分析器

- **递归下降分析器( Recursive-Descent Parser )**

由一组递归过程组成，且每个过程对应文法的一个$V_N$的分析程序。(将一个非终结符的文法规则看作识别非终结符的一个过程)。

- **递归下降分析器对文法要求：**LL(1)文法

- **递归下降分析器的基本构造方法**

对文法的每个非终结符号，都根据其产生式的各个候选式的结构，为其编写一个对应的子程序（或函数），该子程序完成相应的非终结符对应的语法成份的识别和分析任务。

**例4.9** 设有文法$G(E)$：

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' | \varepsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' | \varepsilon$$

$$F \rightarrow (E) | i$$

文法$G(E)$的递归下降语法分析程序如下

主程序

```
E ( )
  { T; E′;}
```

```
E′( )
{
 if (c='+')
{n++; T; E';} }
```

```
F ( )
{if ( c='i' )  n++;
    else  if ( c='(' )
          { n++; E;
              if (c=')' )  n++;
              else error;  }
        else error;  }
```

E→TE′

E′→+TE′|ε

T→FT′

T′→*FT′ |ε

F→( E) | i

```
T ( )
  { F; T';}
```

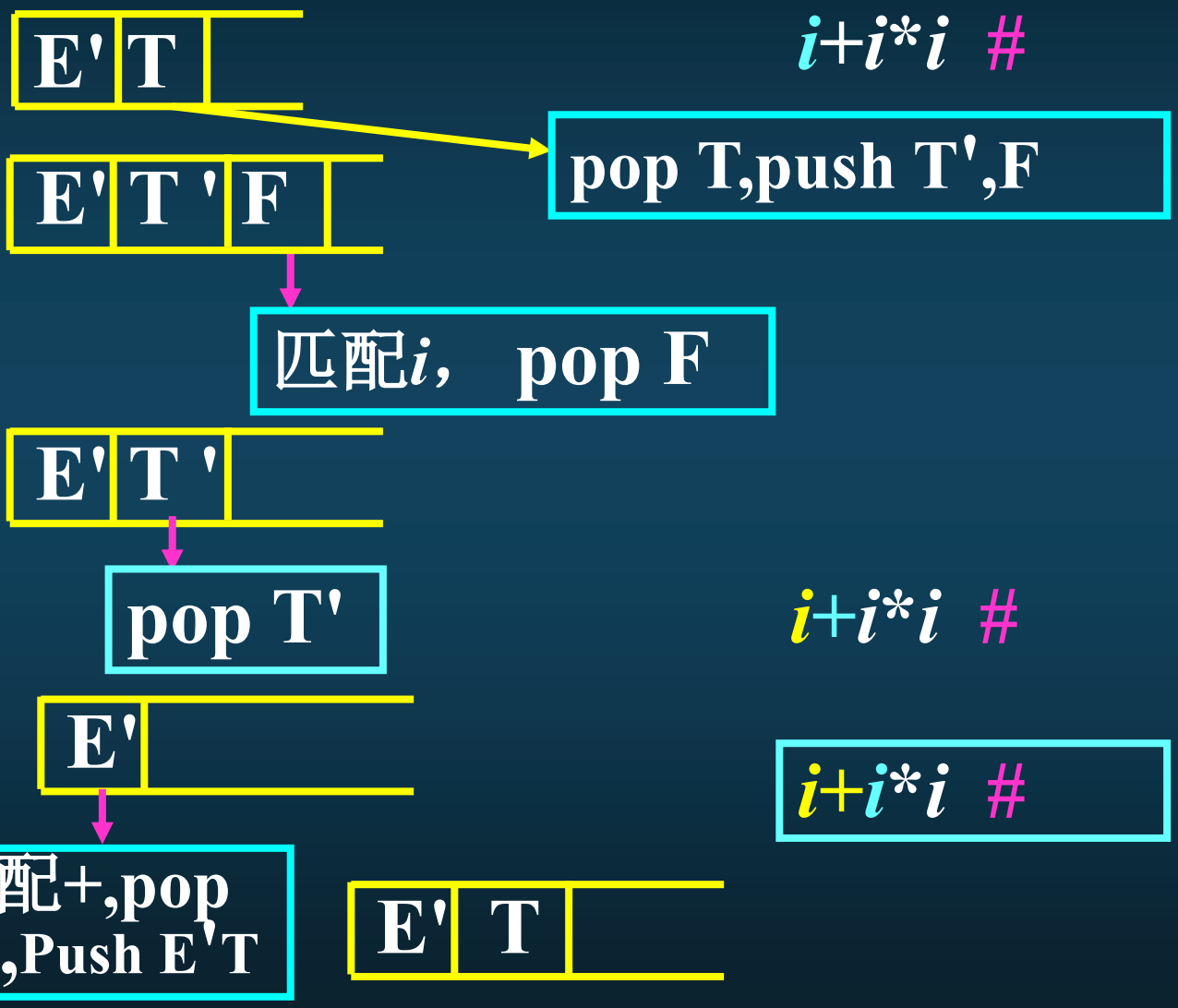
```
T′ ( )
 {  if (c='*')  {n++; F; T′ ;}
 }
```

其中: n — 读单词指针; c —单词指针指的的单词;

查看

设有 $r1=i+i*i$ #

stack




$E \to TE'$

$E' \to +TE' | \varepsilon$

$T \to FT'$

$T' \to *FT' | \varepsilon$

$F \to (E) | i$

查看

| E' | T | |
|---|---|---|

pop T,
push T',F

| E' | T ' | F | |
|---|---|---|---|

匹配i, pop F

| E' | T ' | | |
|---|---|---|---|

匹配*,pop T',Push T',F

| E' | T ' | F | |
|---|---|---|---|

匹配i, pop F

*i*+*i***i* #

*i*+*i***i* #

*i*+*i***i* #

*i*+*i***i* #

| E' | T ' | | |
|---|---|---|---|

pop T',E'

| | | |
|---|---|---|

*i*+*i***i* #

分析完成格局

$E \to TE'$

$E' \to +TE' \mid \varepsilon$

$T \to FT'$

$T' \to *FT' \mid \varepsilon$

$F \to (E) \mid i$

查看

设有 r1=（+i #

$E \to TE'$

$E' \to +TE' \mid \varepsilon$

$T \to FT'$

$T' \to *FT' \mid \varepsilon$

$F \to (E) \mid i$

| E' | T | |
|---|---|---|

Push T 'F

（+i #

| E' | T ' | F | |
|---|---|---|---|

匹配（
pop F,Push ) E

（+i #

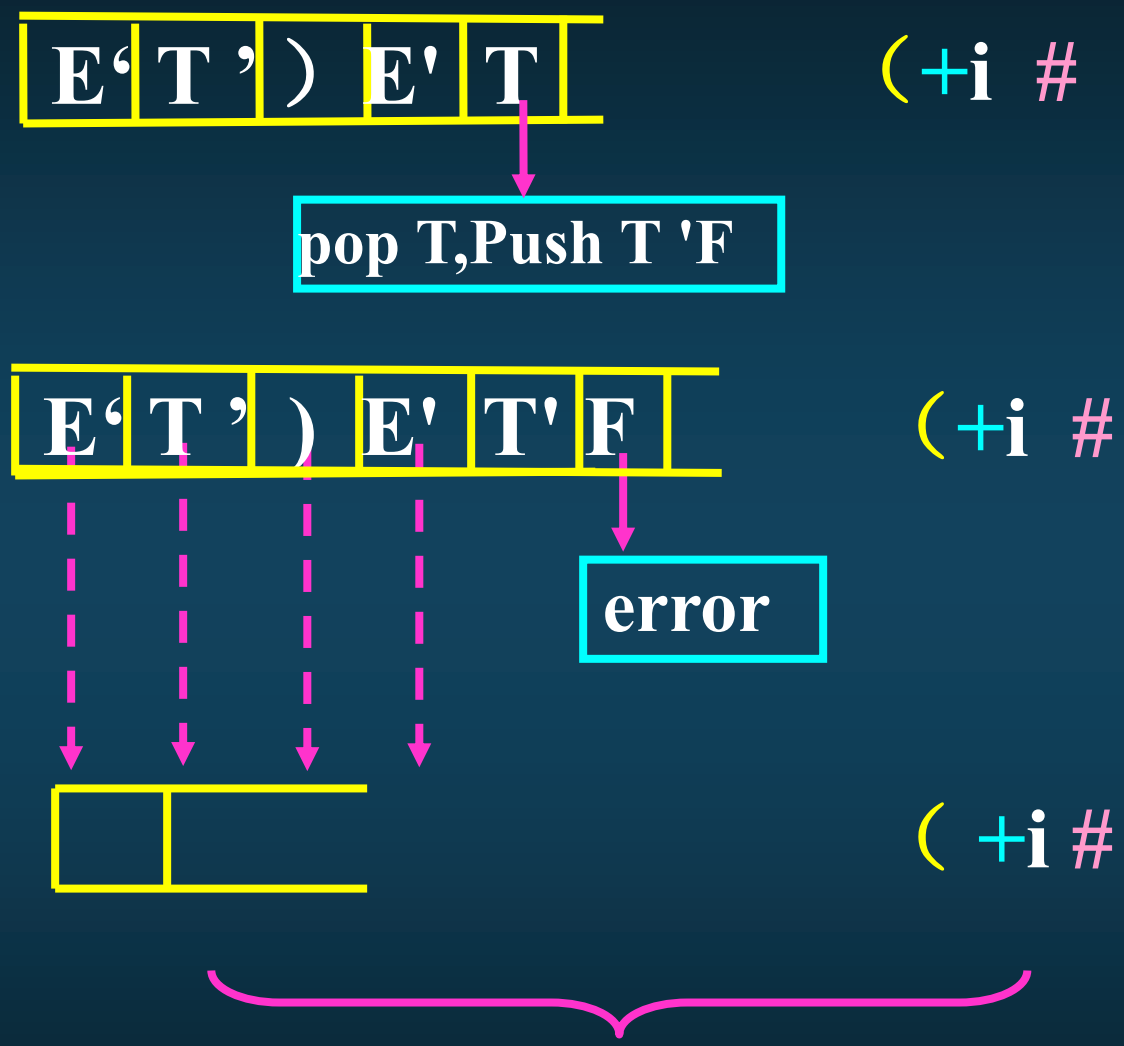
| E' | T ' | ) | E |
|---|---|---|---|

pop E,Push E' T

（+i #

| E' | T ' | ） | E' | T | |
|---|---|---|---|---|---|

查看




$E \to TE'$

$E' \to +TE' \mid \varepsilon$

$T \to FT'$

$T' \to *FT' \mid \varepsilon$

$F \to (E) \mid i$

分析完成格局

## 递归下降分析器设计工具 — 状态图(FA)

- 状态图构造：为文法$G$的每个$V_N$

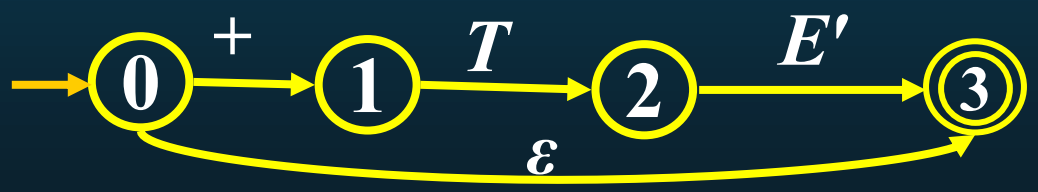
1） 建立1个初态和1个终态(函数返回态)；

2） 为每个产生式 $A \rightarrow x_1x_2 \ldots x_n$ 建立从初态到终态的路径，弧标记为$x_1$，$x_2$，…，$x_n$；

** 其中：$x_i \in (V_T \cup V_N)$

例如，$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \varepsilon$



## 如何利用状态图构造递归下降分析器

1） 开始处于图的初态，即分析器的进入点；

2） 在某状态S，

若 $S \xrightarrow{a} t$ $a \in V_T$

当前输入符号为$a$，分析器扫描指针+1，进入t状态;

若 $S \xrightarrow{A} t$ $A \in V_N$

分析器进入对$A$的分析，扫描指针不变，分析完$A$后进入到状态t；

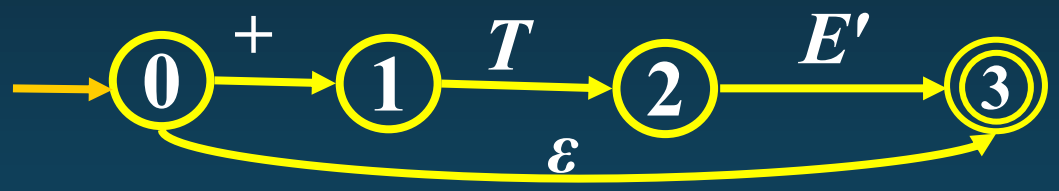
若 $S \xrightarrow{\varepsilon} t$

直接进入t状态，扫描指针不变;

# 通过状态图的化简来化简递归下降分析器

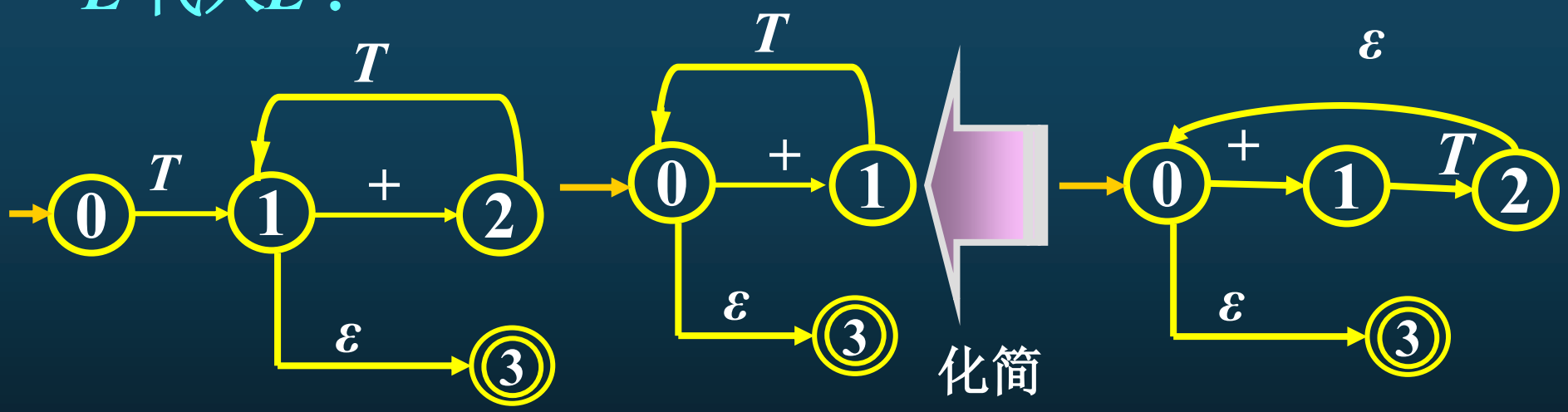
例如，$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \varepsilon$

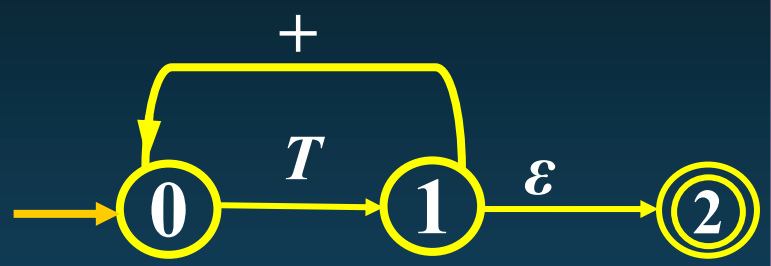



$E'$的递归直接进入$E'$的FA的开始

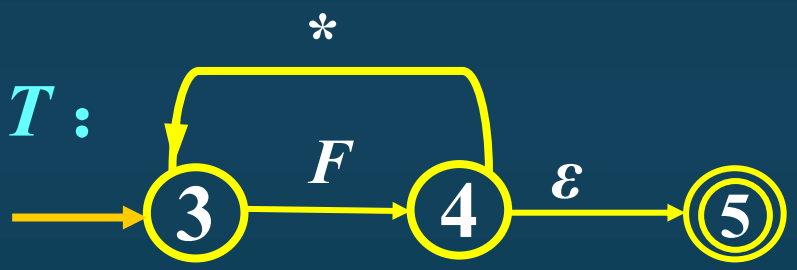
$E'$代入$E$：



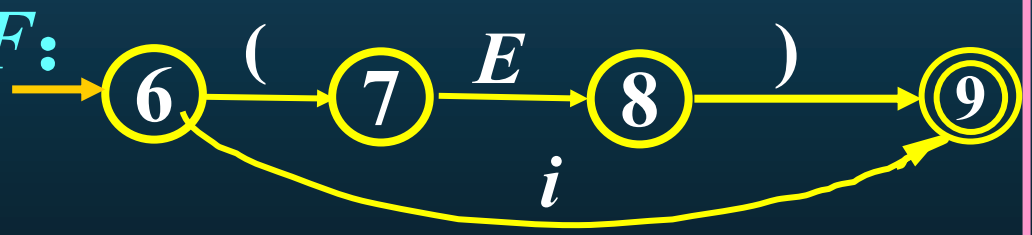
$E'$代入$E$：

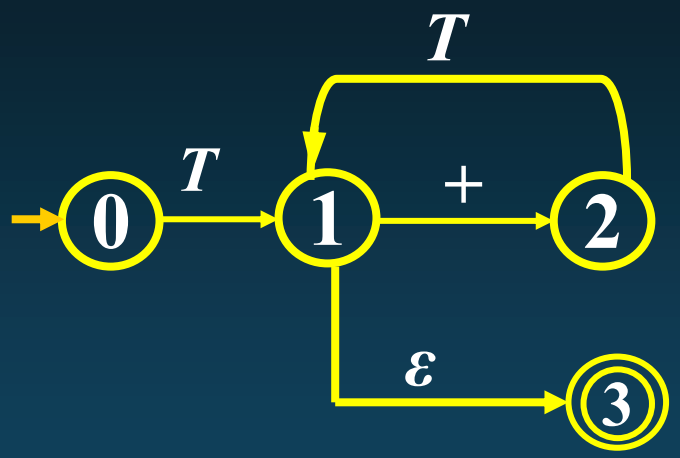



$T'$代入 $T$：




$F$：




$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \varepsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' \mid \varepsilon$

$F \rightarrow (E) \mid i$

# 第 4 章　语法分析(Syntax Analysis)

## —— 自上而下分析法

4.1　语法分析综述

4.2　不确定的自上而下分析法

4.3　递归下降分析法与递归下降分析器

4.4　LL(1)分析法与LL(1)分析器

# 4.4 LL(1)分析法与LL(1)分析器

## 4.4.1 LL(1)分析器的逻辑结构

## 4.4.2 LL(1)分析器的构造

## 4.4.3 关于LL(1)文法

# 预测分析器

**L L (1)**

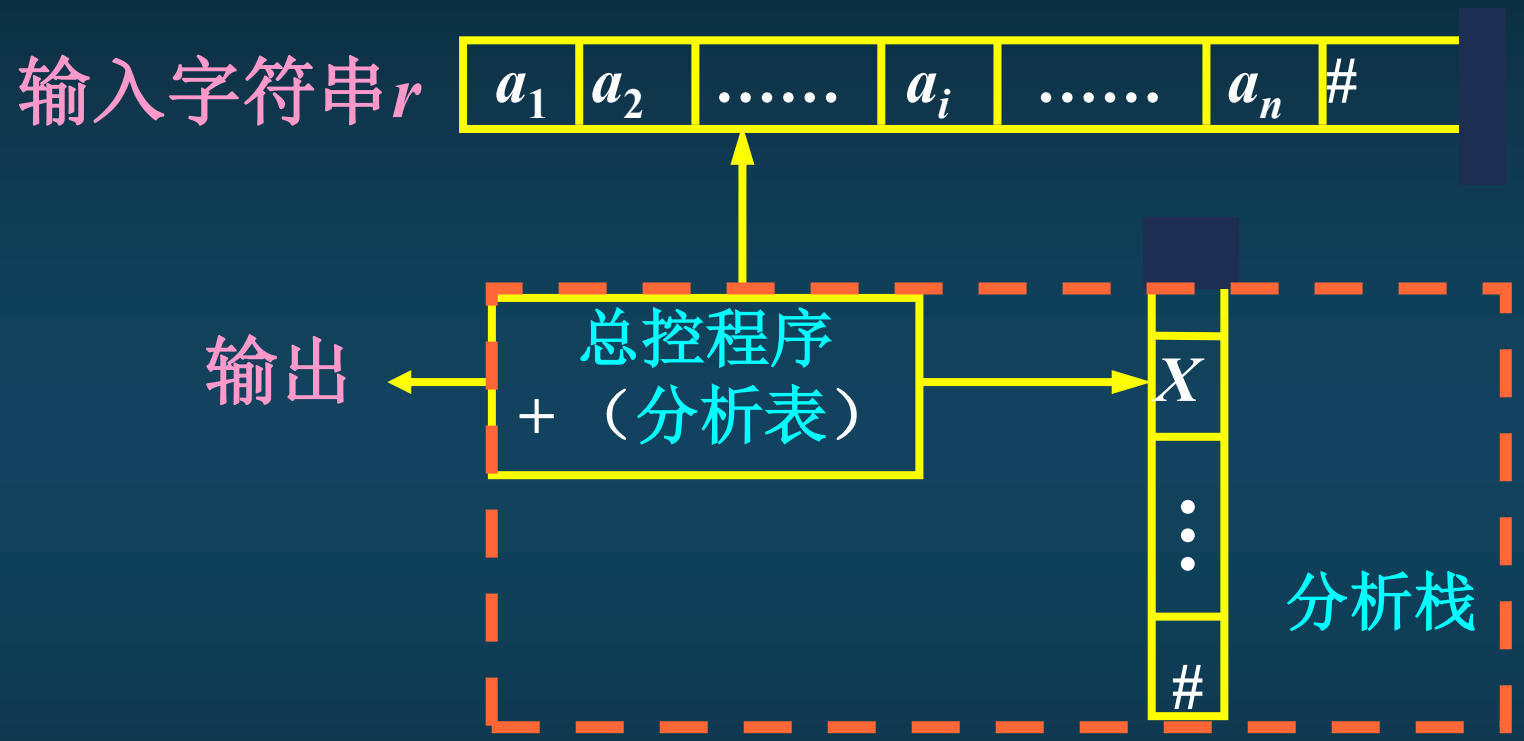
在分析中最多向前看1个输入字符

分析模式：最左推导

扫描模式：自左向右

显式地维护一个状态栈，而不是通过隐式的递归调用来做自上而下的语法分析。

# LL(1)分析器的逻辑结构

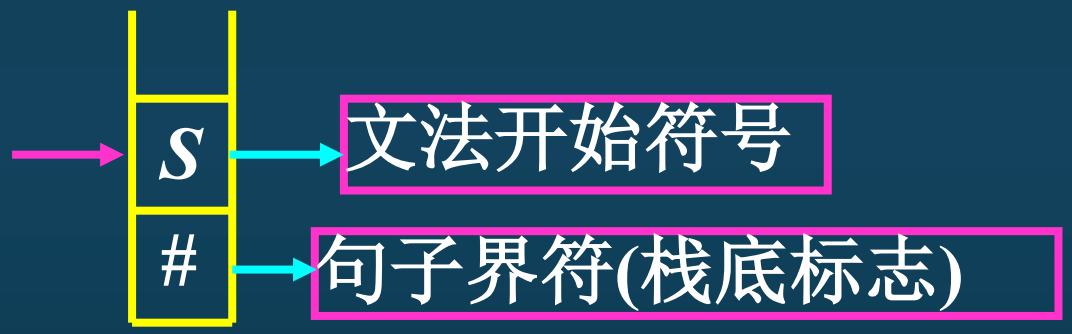



组成
- 总控程序
- LL(1)分析表
- 分析栈

## (1) 分析栈

存放分析过程中的文法符号（已经推导出的待处理的串）。

初始为：



## (2) 分析表

文法的$V_T$

| 文法的$V_N$ | $a_1$ | $a_2$ | … | $a_n$ | # |
|---|---|---|---|---|---|
| $A_1$ | $M(A_1,a_1)$ | $M(A_1,a_2)$ | … | $M(A_1,a_n)$ | |
| $A_2$ | | | | | |
| ⋮ | ⋮ | | | | |
| $A_n$ | | | | | |

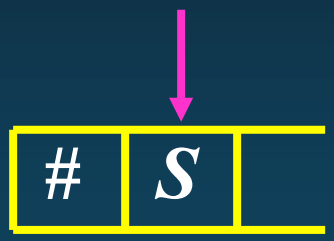
预测分析函数

$M(A_i,a_i)$ { 文法的一条产生式规则(候选式唯一)
出错（空白）

# (3) 总控程序 (LL(1)分析)

- 算法4.2

(1) 初始化工作：



(2) 若当前分析栈顶符号 $X$是文法的终结符号，则对于：

① $X=a_i=$“#”，表示分析成功，停止分析过程；

② $X=a_i\neq$“#”，则将 X 从分析栈顶退掉，p指向下一个输入字符；

③ $X\neq a_i$，表示不匹配的出错情况。

(3) 若 $X \in V_N$，则查分析表。此时对M($X$, $a_i$)：

① 若M( $X$, $a_i$)中为一个产生式规则，则将$X$从栈中弹出并将此规则右部的符号序列按倒序推进栈(若产生式规则为 $X \rightarrow \varepsilon$，则仅将 $X$从栈中弹出) 。

② 若M( $X$, $a_i$)中为空白，表示出错，可调用语法出错处理子程序。

**例4.10** 设有文法$G(E)$：

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' \mid \varepsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' \mid \varepsilon$$

$$F \rightarrow (E) \mid id$$

和文法的LL(1)分析表，对输入串 $id+id*id$ 使用LL(1)分析器的分析过程。

# 文法G(E)的LL(1)分析表

| | + | * | ( | ) | id | # |
|---|---|---|---|---|---|---|
| $E$ | | | $E\to TE'$ | | $E\to TE'$ | |
| $E'$ | $E'\to +TE'$ | | | $E'\to\varepsilon$ | | $E'\to\varepsilon$ |
| $T$ | | | $T\to FT'$ | | $T\to FT'$ | |
| $T'$ | $T'\to\varepsilon$ | $T'\to *FT'$ | | $T'\to\varepsilon$ | | $T'\to\varepsilon$ |
| $F$ | | | $F\to (E)$ | | $F\to id$ | |

| 步 骤 | 分析栈 | 余留输入串 | 所用产生式 |
|---|---|---|---|
| 1 | #E | id+id*id# | E→TE′ |
| 2 | #E′T | id+id*id# | T→FT′ |
| 3 | #E′T′F | id+id*id# | F→id |
| 4 | #E′T′id | id+id*id# | p++ |
| 5 | #E′T′ | +id*id# | T′→ε |
| 6 | #E′ | +id*id# | E′→+TE′ |
| 7 | #E′T+ | +id*id# | p++ |
| 8 | #E′T | id*id# | T→FT′ |
| 9 | #E′T′F | id*id# | F→id |
| 10 | #E′T′id | id*id# | p++ |
| 11 | #E′T′ | *id# | T′→*FT′ |

表

| 步 骤 | 分析栈 | 余留输入串 | 所用产生式 |
|---|---|---|---|
| 12 | # E ′ T′ F* | *id# | p++ |
| 13 | # E ′ T′ F | id# | F→id |
| 14 | # E ′ T′ id | id# | p++ |
| 15 | # E ′ T′ | # | T ′→ε |
| 16 | # E ′ | # | E′→ε |
| 17 | # | # | 分析成功 |

**注意：**

(1) 整个分析过程是分析栈和待匹配串构成的二元式不断变化的过程。

(2)不同的源语言仅是分析表不同，分析器结构、总控程序不变。

# 4.4 LL(1)分析法与LL(1)分析器

## 4.4.1 LL(1)分析器的逻辑结构

## 4.4.2 LL(1)分析器的构造

## 4.4.3 关于LL(1)文法

LL(1)分析器构造关键 → 分析表的构造

↓

分析表的构造关键 → 预测函数

↓

根据LL(1)分析过程，问题的关键是解决根据当前要替换的文法的非终结符和下一步要匹配的终结符，选择非终结符对应的产生式规则的哪一个候选式。

# 第一种情况：

据FIRST集合的定义：对文法$G$，若$G$中产生式形如$A\rightarrow\alpha$且没有 $\alpha$=>ε的情况，则产生LL(1)分析表的预测函数：

对文法规则 $A\rightarrow\alpha$

若$a\in$FIRST($\alpha$)，那么要匹配输入单词$a$时，应该使用候选式$\alpha$替换$A$。所以有

$$M(A, a) = \{A\rightarrow\alpha\}$$

$S \to Ap \mid Bq$ $A \to a \mid cA$ $B \to b \mid dB$

对$S$：FIRST($Ap$)={$a$,$c$} 对$A$：FIRST($a$)={$a$} 对$B$：FIRST($b$)={$b$}

FIRST($Bq$)={$b$,$d$} FIRST($cA$)={$c$} FIRST($dB$)={$d$}

## 文法G的LL(1)分析表

| | $a$ | $b$ | $c$ | $d$ | $p$ | $q$ |
|---|---|---|---|---|---|---|
| $S$ | $S\to Ap$ | $S\to Bq$ | $S\to Ap$ | $S\to Bq$ | | |
| $A$ | $A\to a$ | | $A\to cA$ | | | |
| $B$ | | $B\to b$ | | $B\to dB$ | | |

若有 $\varepsilon \in$FIRST($\alpha$)怎么办？此时当面临 $a \notin$ FIRST($\alpha$)时并不一定出错。

- **定义4.5**

设上下文无关文法$G$，$S$是文法的开始符号，对于文法$G$的任何非终结符$A$

$$\mathrm{FOLLOW}(A)=\{a \mid S \overset{*}{\Rightarrow} \ldots Aa \ldots,\ a \in V_T\}$$

若$S \overset{*}{\Rightarrow} \ldots A$，则令$\# \in \mathrm{FOLLOW}(A)$。

FOLLOW($A$)的含义是指，在文法$G$的句型中，能够紧跟着$A$之后的一切终结符或“#”。

- 构造FOLLOW集方法
- 算法4.3

文法$G$中的每一个$B\in V_N$，为构造FOLLOW($B$)，可反复应用如下规则：

① 对文法的开始符号$S$，令#∈FOLLOW($S$)；

② 若文法$G$中有形如$A\rightarrow\alpha B\beta$的规则，且$\beta\neq\varepsilon$，则将FIRST($\beta$)中的一切非$\varepsilon$符号加入FOLLOW($B$)；

③ 若$G$中有形如$A\rightarrow\alpha B$或$A\rightarrow\alpha B\beta$的规则，且$\varepsilon\in$FIRST($\beta$)，则FOLLOW($A$)中的全部元素属于FOLLOW($B$)。

**例4.11** 设有文法$G[S]$为：

$S \rightarrow AB \mid bC$ $A \rightarrow \varepsilon \mid b$

$B \rightarrow \varepsilon \mid aD$ $C \rightarrow AD \mid b$

$D \rightarrow aS \mid c$

据算法4.3计算文法$G[S]$所有$V_N$的FOLLOW集。

$S \to AB \mid bC$

$A \to \varepsilon \mid b$

$B \to \varepsilon \mid aD$

$C \to AD \mid b$

$D \to aS \mid c$

$\# \to$ FOLLOW($S$);

产生式：$S \to AB$ ：

FOLLOW($S$)$\subseteq$ FOLLOW($B$) ;

FOLLOW($S$) $\subseteq$ FOLLOW($A$);

FIRST($B$)$-\{\varepsilon\} \subseteq$ FOLLOW($A$);

产生式：$S \to bC$ ：

FOLLOW($S$) $\subseteq$ FOLLOW($C$);

产生式$B \to aD$得::

FOLLOW($B$) $\subseteq$ FOLLOW($D$)

产生式$C \to AD$得:

FOLLOW($C$) $\subseteq$ FOLLOW($D$)，FIRST($D$) $-\{\varepsilon\} \subseteq$ FOLLOW($A$) ,

产生式$D \to aS$ 得:

FOLLOW($D$) $\subseteq$ FOLLOW($S$),

所以FOLLOW(S)=FOLLOW(B)=FOLLOW(D)=FOLLOW(C)={#}

FOLLOW($A$)={#, $a$, $c$};

# 2. 构造FOLLOW集方法二——关系图法

(1)文法$G$中的每个终结符和“#”对应图中的一个结点，每个非终结符$A$对应两个结点FOLLOW($A$)和FIRST($A$)。

(2)从开始符号$S$的FOLLOW($S$)结点到”#”号的结点连一条箭弧。

(3)如果文法中有产生式$A\rightarrow\alpha BX$,当$X\in V_N$时，则从FOLLOW($B$)结点到FIRST($X$)结点连一条弧，当$X\in V_T$时，则与$X$相连。

(4)如果文法中有产生式$A\rightarrow \alpha B\beta$,且$\beta\overset{*}{\Rightarrow}\varepsilon$,则从FOLLOW($B$)结点到FOLLOW($A$) 结点连一条箭弧。

(5)对每一FIRST($A$)结点如果有产生式$A\rightarrow X_1X_2\ldots X_k$,当$X_1\in V_T$时,则与$X_1$相连;当$X_1\in V_N$时,从FIRST($A$)到FIRST($X_1$)结点连一条箭弧,当$X_1X_2\ldots X_{i-1}\overset{*}{\Rightarrow}\varepsilon$，从FIRST($A$)到FIRST($X_i$)结点连一条箭弧。

(6)凡是从FOLLOW($A$)结点有路径可以到达的终结符或“#”号的结点，其所标记的终结符或“#”号即为FOLLOW($A$)的成员。

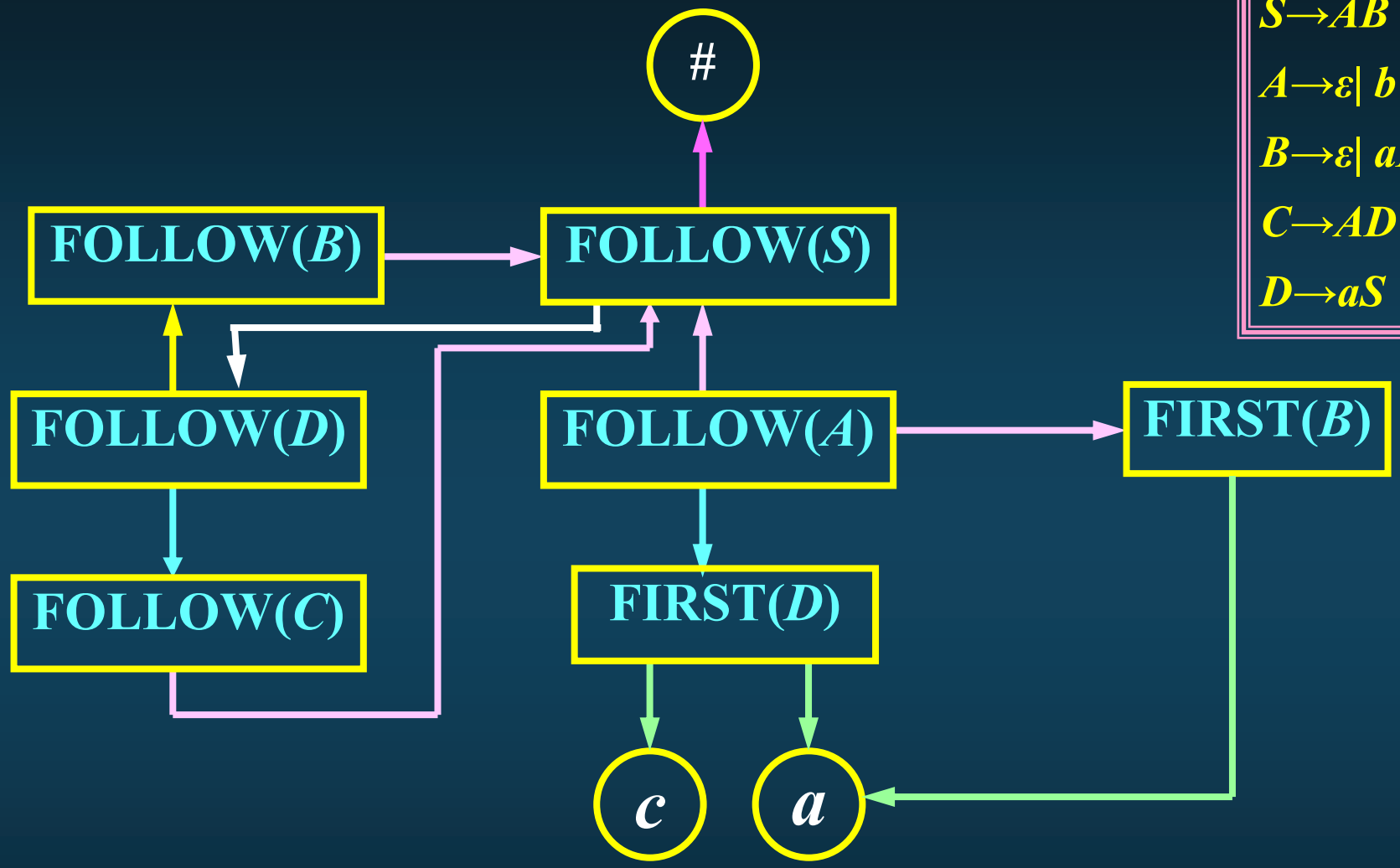
$S \rightarrow AB \mid bC$

$A \rightarrow \varepsilon \mid b$

$B \rightarrow \varepsilon \mid aD$

$C \rightarrow AD \mid b$

$D \rightarrow aS \mid c$




FOLLOW(*S*)=FOLLOW(*B*)=FOLLOW( *D*)=FOLLOW (*C*) ={#}；

FOLLOW(*A*)={#，*a*，*c*}；

自上而下语法分析的主旨是选择产生式的合适的侯选式进行推导，逐步使推导结果与$r$匹配。

LL(1)分析法用分析栈和输入缓冲区记录分析的过程，分析表为根据当前情况确定选择那一个候选式的表格。

所以分析表的构造：

（1）对文法$G$，若$G$中对非终结符$A$的产生式形如$A\rightarrow\alpha|\beta$且 $\alpha$和$\beta$均不推出$\varepsilon$，则可产生LL(1)分析表的$A$行为，

对$\forall\ a\in$ FIRST($\alpha$)，$a$列即 $M(A,\ a)=\{A\rightarrow\alpha\}$

对$\forall\ b\in$ FIRST($\beta$)，$b$列即 $M(A,\ b)=\{A\rightarrow\beta\}$

(2)对$A\rightarrow\alpha|\beta$且 $\alpha$或$\beta$推出$\varepsilon$的情况，产生LL(1)分析表的$A$行为，

若$\alpha \overset{*}{\Rightarrow} \varepsilon$

对$\forall\ a\in$FIRST($\alpha$)$\cup$FOLLOW($A$)，

$a$列即 $M(A, a) = \{A\rightarrow\alpha\}$

若$\alpha$不推出空串，

对$\forall\ a\in$FIRST($\alpha$)，$a$列即 $M(A, a) = \{A\rightarrow\alpha\}$

若$\beta \overset{*}{\Rightarrow} \varepsilon$

对$\forall\ b\in$FIRST($\beta$)$\cup$FOLLOW($A$)，

$b$列即 $M(A, b) = \{A\rightarrow\beta\}$

若$\beta$不推出空串，

对$\forall\ b\in$FIRST($\alpha$)，$b$列即 $M(A, b) = \{A\rightarrow\beta\}$

# 分析

已知 FOLLOW($A$)＝{$a$ | $S \overset{*}{\Rightarrow} \cdots Aa \cdots$，$a \in V_T$}

若有 $b \in$ FOLLOW($A$)则必有

$\cdots Ab \cdots (b \in V_T)$这样的句型。

设 $S \overset{*}{\Rightarrow} \cdots aAb \cdots$ 由于对文法规则存在$A \rightarrow \alpha$且$\varepsilon \in$ FIRST($\alpha$)，当$\alpha \overset{*}{\Rightarrow} \varepsilon$，则$A \overset{*}{\Rightarrow} \varepsilon$。

故有$S \overset{*}{\Rightarrow} \cdots aAb \cdots \Rightarrow \cdots \underline{ab} \cdots$

要获得匹配

$\therefore$ M($A$ , $b$) $= A \rightarrow \alpha$（这样在自上而下分析的推导中才可能从栈中退掉$A$。）

- **算法4.4 LL(1)分析表构造**

**输入：** 文法$G$；$G$候选式的FIRST、候选式FIRST有$\varepsilon$的非终结符号的FOLLOW集合

**输出：** 文法$G$的LL(1)分析表

**方法：** for 文法$G$的每个产生式$A \to \gamma_1 | \gamma_2 | \cdots | \gamma_m$

{ if $a \in$ FIRST ($\gamma_i$) 置$M(A, a) = A \to \gamma_i$；

if $\varepsilon \in$ FIRST ($\gamma_i$)

for 任何 $b \in$ FOLLOW($A$)

{ 置$M(A, b) = A \to \gamma_i$ }

else 置所有无定义的$M(A, a_i)$为出错

}

**前例4.10 设有文法$G(E)$：**

$$E \rightarrow TE'$$
$$E' \rightarrow +TE' \mid \varepsilon$$
$$T \rightarrow FT'$$
$$T' \rightarrow *FT' \mid \varepsilon$$
$$F \rightarrow (E) \mid i$$

**对E：** FIRST($TE'$)={ (, $i$ }

**对T：** FIRST( $FT'$ )={ (, $i$ }

**对F：** FIRST(($E$))={ ( } FIRST( $i$ )={ $i$ }

$E \rightarrow TE'$

$E' \rightarrow +TE' | \varepsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' | \varepsilon$

$F \rightarrow (E) | i$

对$E'$：

FIRST(+ $TE'$ )={+}

FIRST($\varepsilon$)= { $\varepsilon$ }

FOLLOW($E'$ )= { #, ) }

对T′：

FIRST(*$FT'$ )={ * }

FIRST($\varepsilon$ )={ $\varepsilon$ }

FOLLOW($T'$ )={ #, ), + }

$E \to TE'$: FIRST($TE'$)={(, $i$}

$E' \to +TE' | \varepsilon$:FIRST($+TE'$)={+} FIRST($\varepsilon$)={$\varepsilon$} FOLLOW($E'$)= { #, ) }

$T \to FT'$: FIRST( $FT'$ )={(, $i$}

$T' \to *FT' | \varepsilon$:FIRST($*FT'$)={*} FIRST($\varepsilon$)={$\varepsilon$} FOLLOW($T'$)={ #, ),+ }

$F \to (E) | i$ :FIRST(($E$))={(} FIRST( $i$ )={$i$}

## 文法$G(E)$的LL(1)分析表

| | + | * | ( | ) | $i$ | # |
|---|---|---|---|---|---|---|
| $E$ | | | $E \to TE'$ | | $E \to TE'$ | |
| $E'$ | $E' \to +TE'$ | | | $E' \to \varepsilon$ | | $E' \to \varepsilon$ |
| $T$ | | | $T \to FT'$ | | $T \to FT'$ | |
| $T'$ | $T' \to \varepsilon$ | $T' \to *FT'$ | | $T' \to \varepsilon$ | | $T' \to \varepsilon$ |
| $F$ | | | $F \to (E)$ | | $F \to i$ | |

设有 $r1=(+i\ \#$

$E\rightarrow TE'$

$E'\rightarrow +TE'|\varepsilon$

$T\rightarrow FT'$

$T'\rightarrow *FT'|\varepsilon$

$F\rightarrow (E)\,|\,i$

| E' | T | |
|---|---|---|

$(+i\ \#$

Push $T'F$

| E' | T' | F | |
|---|---|---|---|

匹配（
pop $F$,Push ) $E$

| E' | T' | ) | E |
|---|---|---|---|

$(+i\ \#$

pop $E$,Push $E'\ T$

| E' | T' | ) | E' | T |
|---|---|---|---|---|

pop $T$,Push $T'F$

$(+i\ \#$

| E' | T' | ) | E' | T' | F |
|---|---|---|---|---|---|

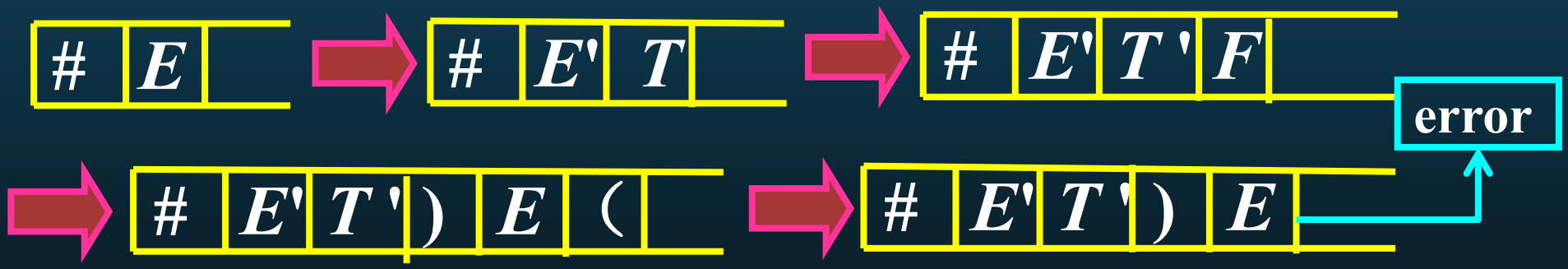
error

$(+i\ \#$

# 文法G(E)的LL(1)分析表

| | + | * | ( | ) | i | # |
|---|---|---|---|---|---|---|
| E | | | E→TE′ | | E→TE′ | |
| E′ | E′→+TE′ | | | E′→ε | | E′→ε |
| T | | | T→FT′ | | T→FT′ | |
| T′ | T′→ε | T′→*FT′ | | T′→ε | | T′→ε |
| F | | | F→(E) | | F→i | |

设有 r1=（+i #



**例4.12** 设有文法$G(S)$

$$S \rightarrow iCtSS' \mid a$$

$$S' \rightarrow eS \mid \varepsilon$$

$$C \rightarrow b$$

对$S$： FIRST( $iCtSS'$ )={ $i$ }

FIRST( $a$ )={ $a$ }

对$S'$： FIRST($eS$ )={ $e$ }

FIRST( $\varepsilon$ )={ $\varepsilon$ } FOLLOW($S'$)={#,$e$}

对$C$： FIRST( $b$ )={ $b$ }

$S \to iCtSS' \mid a$: FIRST( $iCtSS'$ )={ $i$ } FIRST( $a$ )={ $a$ }

$S' \to eS \mid \varepsilon$: FIRST($eS$ )={$e$} FIRST($\varepsilon$)={$\varepsilon$} FOLLOW($S'$)={#,$e$}

$C \to b$: FIRST( $b$ )={ $b$ }

## 文法G(S)的LL(1)分析表

| | $a$ | $b$ | $e$ | $i$ | $t$ | # |
|---|---|---|---|---|---|---|
| $S$ | $S \to a$ | | | $S \to iCtSS'$ | | |
| $S'$ | | | $S' \to eS$<br>$S' \to \varepsilon$ | | | $S' \to \varepsilon$ |
| $C$ | | $C \to b$ | | | | |

多重定义

# 4.4 LL(1)分析法与LL(1)分析器

## 4.4.1 LL(1)分析器的逻辑结构

## 4.4.2 LL(1)分析器的构造

## 4.4.3 关于LL(1)文法

- **定义4.6**

一部文法$G$，若它的LL(1)分析表$M$不含多重定义入口，则称它是一个LL(1)文法。由LL(1)文法产生的语言称为LL(1)语言。

**L L ($k$)**

在分析中最多向前看$k$个输入字符

分析模式：最左推导

扫描模式：自左向右

# 关于LL(1)文法及LL(1)语言重要的性质

⑴任何LL(1)文法是无二义性的。

⑵若一文法为左递归文法，则它必然是非LL(1)文法。

⑶ 非LL(1)语言是存在的。

⑷存在一种算法，它能判定任一文法是否为LL(1)文法。

⑸不存在这样的算法，它能判定上下文无关语言能否由LL(1)文法产生。

非左递归文法$G$为LL(1)文法$\Leftrightarrow G$的任何一个非终结符 $A$，设关于$A$的产生式为

$$A \rightarrow \alpha_1 \mid \alpha_2 \mid \ldots \mid \alpha_n$$

满足下面的条件:

1）每个候选式$\alpha_i$, FIRST($\alpha_i$)两两彼此互不相交。

2）若有$\alpha_i \Rightarrow \varepsilon$，则FIRST($\alpha_i$)∪FOLLOW($A$)与其他候选式的FIRST互不相交。

4－12：试判断下面哪些文法是LL(1)的？如果不是，哪些能改写为LL(1)文法并改写。

（2） $S \rightarrow AB$

$A \rightarrow Ba \mid \varepsilon$

$B \rightarrow Db \mid D$

$D \rightarrow d \mid \varepsilon$

答：该文法显然不是LL(1)文法，因为非终结符$B$的两个候选式存在左公因子。需要提出左公因子。

$B \rightarrow Db \mid D$ 改写为 $B \rightarrow DB'$ $B' \rightarrow b \mid \varepsilon$

文法改写为：

# 第四章习题讲解（4－12(2)）

$S \to AB$

$A \to Ba|\varepsilon$

$B \to DB'$

$B' \to b|\varepsilon$

$D \to d|\varepsilon$

对产生式$A \to Ba|\varepsilon$

First($Ba$)={$d,b,a$},

Follow($A$)=First($B$)∪Follow($S$)－{$\varepsilon$}

={$d,b$}∪{#}={$d,b$,#}

First($Ba$)∩Follow($A$)≠$\Phi$

修改后的文法还不是LL(1)文法。

继续修改，把$A$的产生式代入$S$的候选式，得

$S \to BaB \mid B$

$B \to DB'$

$B' \to b|\varepsilon$

$D \to d|\varepsilon$

提取$S$产生式的公因子：

$S \to BA$

$A \to aB \mid \varepsilon$

# 第四章习题讲解（4－12(2)）

得文法

$S \to BA$

$A \to aB \mid \varepsilon$

$B \to DB'$

$B' \to b \mid \varepsilon$

$D \to d \mid \varepsilon$

修改后的文法为LL(1)文法。

验证：

对$A \to aB \mid \varepsilon$

First($aB$)={$a$};Follow($A$)=Follow($S$)={#}

First($aB$)∩Follow($A$)=$\Phi$

对$B' \to b \mid \varepsilon$

First($b$)={$b$}; Follow($B'$)=Follow($B$)

=First($A$)∪Follw($S$)∪Follw($A$) －{$\varepsilon$}

={$a$}∪{#}∪{＃}={$a$,#}

First($b$)∩Follow($B'$)=$\Phi$

对$D \to d \mid \varepsilon$

First($d$)={$d$};Follow($D$)=

First($B'$)∪Follw($B$) －{$\varepsilon$}={$b$}∪{$a$,#}={$a$,$b$,#}

First($d$)∩Follow($D$)=$\Phi$

$S \rightarrow iCtSS' \mid a$

$S' \rightarrow eS \mid \varepsilon$

$C \rightarrow b$

# 文法G(S)的分析表

| | a | b | e | i | t | # |
|---|---|---|---|---|---|---|
| S | $S \rightarrow a$ | | | $S \rightarrow iCtSS'$ | | |
| S′ | | | $S' \rightarrow eS$ | | | $S' \rightarrow \varepsilon$ |
| C | | $C \rightarrow b$ | | | | |

# 本章提要

- ✓语语分析器的任务。
- ✓ 语法分析的主要方法：自上而下和自下而上分析。
- ✓ 确定的自上而下分析的条件：*G*不含左递归、不含左公因子。
- ✓递归下降分析器的构造原理与实现。
- ✓LL(K)及LL(1)分析的实现思想。
- ✓LL(1)分析表与LL (1)分析器的构造。
- ✓LL (1)文法的判别。

# end

(1) 语法分析方法中的LL(1)分析法属于_____分析方法。

A）自左至右　　B）自上而下

C）自下而上　　D）自右至左

(2) 下列文法中，_____是LL(1)文法。（$S$是公理）

A) $S \to aSb \mid ab$　　B) $S \to ab \mid Sab$

C) $S \to aS \mid b$　　D) $S \to aS \mid a$

(3)设有文法$G[S]$为：

$S \to AB \mid bC$　　$A \to \varepsilon \mid b$　　$B \to \varepsilon \mid aD$

$C \to AD \mid b$　　$D \to aS \mid c$

则FOLLOW($A$) = {$a, \#, c$}，FIRST($S$) ={$b, a, \varepsilon$}。

(4) 设有文法$G(S)$： $S \to Pab \mid bP$ $P \to b \mid \varepsilon$

根据文法$G(S)$，填写如下LL(1)分析表的内容。

| | $a$ | $b$ | # |
|---|---|---|---|
| $P$ | $P \to \varepsilon$ | $P \to b$ | $P \to \varepsilon$ |

(5) 设有文法$G$（$S$为开始符号）：

$$S \to Ap \mid Bq$$

$$A \to a \mid cA$$

$$B \to b \mid dB$$

FIRST($Ap$) = {_____}

A) $a$，$c$ B) $b$，$d$ C) $p$，$q$ D) 其他答案

判断正误。

(1) 语法分析的任务是分析语句是如何构成程序的。×

(2) 设有文法G： $S \rightarrow qQ \mid q$ $Q \rightarrow cQd \mid \varepsilon$，该文法是LL(1)文法。×

(3) LL(1)分析法中的“1”是指扫描源程序当前指针P++所指的源程序串的字符。×

(4) 自上而下分析及自下而上分析中的“下”指的是被分析的源程序串。√

(5) 语法分析方法中的递归下降分析法属于自上向下分析方法。√